Windows AppLocker & Group Policy Security Lab

Application Control with Path, Publisher and File Hash Rules

Valentin Ochioiu
Network Security Student

Folkuniversitetet, Gothenburg
5 February 2026

Project Overview

Windows application control using AppLocker policy

The lab focused on controlling which applications a Windows user could execute. AppLocker was tested with several rule conditions instead of one broad allow-list.

Implemented

- Default Windows and administrator rules
- User-specific path-based Deny rule for Notepad++
- Restrictive allow-listing with file-hash rules
- Publisher, Path and Hash conditions in one policy
- Windows Installer, Script and Packaged App rule collections
- Functional testing and AppLocker event-log verification

Core security idea

Only applications matching approved rules should execute. Rule scope can depend on identity, path, publisher signature or exact file identity.

Scope

This presentation covers only the Windows AppLocker and Group Policy work from the lab, using local rules and standard-user execution tests.

Test Model

Windows test system

Windows Server 2022 was the AppLocker test host. Rules were configured through Local Group Policy, with Application Identity enabled for enforcement.

Identity model

Administrator access was preserved. A dedicated **TestUser** was used to validate restrictions from a standard-user perspective.

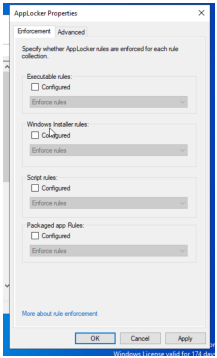
Test cycle

Windows host: 192.168.122.20. After policy changes, run `gpupdate /force`, launch applications as TestUser and inspect AppLocker events.

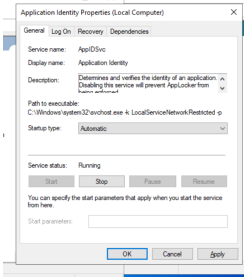
How the policy was tested

1. **Safe baseline** — Create Windows and Administrator default rules to preserve system operation and recovery.
2. **User-specific deny** — Block Notepad++ for TestUser with a Path rule and verify the failure in AppLocker logs.
3. **Restrictive allow-list** — Remove the broad Program Files allowance and allow selected software by hash.
4. **Multi-tier policy** — Combine Publisher, controlled Path and File Hash rules and test the result.

AppLocker Configuration and Enforcement



Initial configuration view, before selecting collection enforcement.

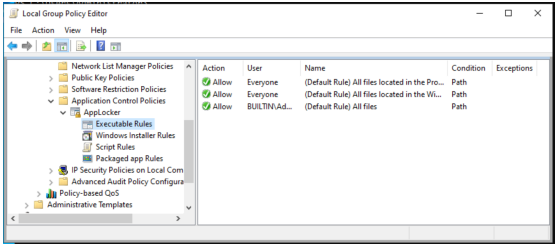


Application Identity service properties used during testing.

Implemented

Application Identity is running. Later execution tests and block events demonstrate enforcement. The setup capture at left still shows unselected configuration boxes.

Default Rules and an Important Failure Case



Default executable rules retained for Windows and administrators.

Why they mattered

The Windows-folder rule preserved essential system execution, while the administrator rule kept full administrative access during testing.

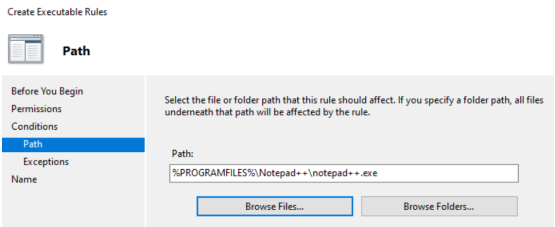
Failure case

Removing the Windows default rule blocked important system files. The shell failed to start and recovery required Safe Mode and disabling AppLocker.

Lesson learned

- Default-deny can affect the operating system itself.
- Trusted system execution must be preserved explicitly.
- Restrictive policies should be introduced incrementally.

User-Specific Path Rule — Notepad++



Path condition configured for the Notepad++ executable.

Policy objective

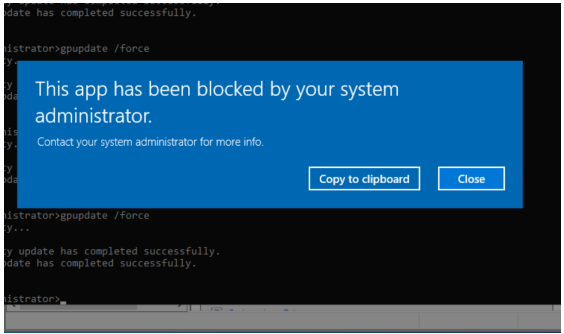
Block Notepad++ for one user rather than for every account on the system.

- Created a dedicated **TestUser**.
- Added a path-based **Deny** rule.
- Scoped the rule to TestUser.
- Retained the Windows defaults for stability.

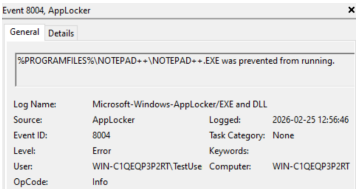
Security value

AppLocker can enforce different application rights for different user identities on the same Windows system.

Path Rule Validation — Blocked Execution



Observed result: the application was blocked by system policy.

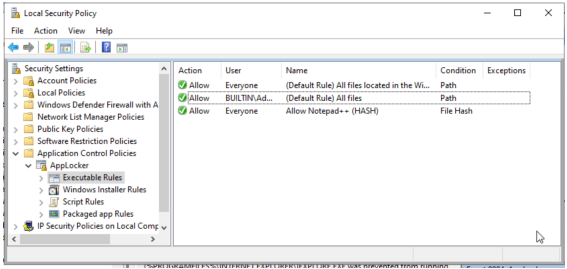


AppLocker event recording the prevented execution.

Validated

The application was actually prevented from running and the AppLocker log recorded the enforcement event.

Hash-Based Rule Under a More Restrictive Policy



Executable rules after removing the broad Program Files allowance.

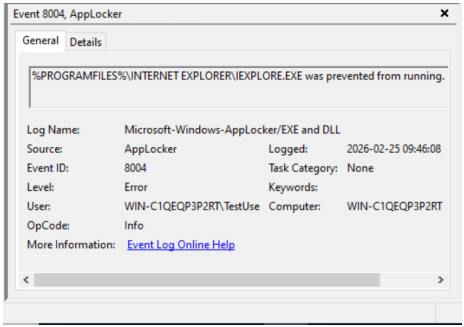
Policy change

- Administrator access to all files was retained.
- The broad *Allow Everyone — Program Files* rule was removed.
- A hash rule allowed one specifically approved executable.
- The Windows rule remained for system stability.

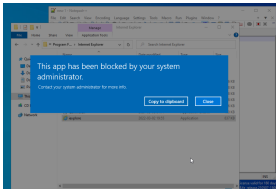
Default-deny effect

Once the broad Program Files allow rule was removed, software without another matching Allow rule was blocked automatically.

Hash Rule Validation and Default-Deny Behaviour



AppLocker Event Viewer evidence from the restrictive-rule test.



Internet Explorer shown as one blocked example.

Observed behaviour

Internet Explorer was not special-cased. It was simply one visible example of expected default-deny behaviour: after the broad Program Files Allow rule was removed, other Program Files applications were also blocked unless another Allow rule matched.

Multi-Tier Application Control Policy

Action	User	Name	Condition
Allow	Everyone	mIRC version 7.83.0.0 only	Publisher
Allow	Everyone	Allow * to SpecialApps	Path
Allow	Everyone	(Default Rule) All files located in the Windows folder	Path
Allow	BUILTIN\Ad...	(Default Rule) All files	Path
Allow	Everyone	Hash Verification CPU-Z	File Hash

Combined executable rule set using multiple AppLocker condition types.

Publisher

Allowed signed mIRC using publisher and version criteria.

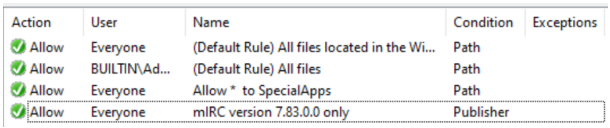
Path

Allowed programs stored in the designated SpecialApps directory.

File hash

Allowed one exact CPU-Z executable by file identity.

Publisher Rule — Signed Application Control



Action	User	Name	Condition	Exceptions
Allow	Everyone	(Default Rule) All files located in the Wi...	Path	
Allow	BUILTIN\Ad...	(Default Rule) All files	Path	
Allow	Everyone	Allow * to SpecialApps	Path	
Allow	Everyone	mIRC version 7.83.0.0 only	Publisher	

Executable policy including the mIRC Publisher rule.

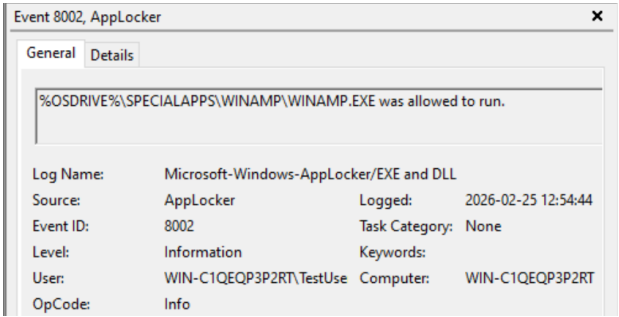
Implemented

- Publisher condition referencing mIRC version 7.83.0.0.
- Trust based on digital-signature identity rather than only file location.

Why use it?

Publisher rules are useful for signed software when policy should follow publisher, product or version information.

Path Rule — Controlled SpecialApps Directory



Event 8002: Winamp in SpecialApps was allowed for TestUser.

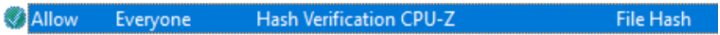
Implemented rule

Programs stored under the designated **SpecialApps** path were allowed. Software outside approved conditions remained subject to default-deny.

Security consideration

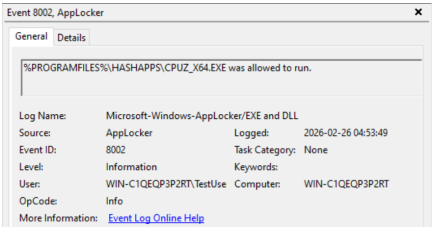
A path rule is only as trustworthy as the permissions protecting that directory. A user-writable allowed path can become a route for unapproved software.

File Hash Rule – CPU-Z Evidence

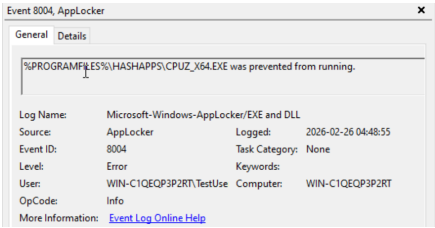


The policy identifies “Hash Verification CPU-Z” with condition File Hash.

Allowed execution: Event 8002



Blocked execution: Event 8004



Hash value is not visible in the lab captures

The ZIP shows the File Hash rule and CPU-Z execution events, but no hash-condition dialog or raw value. The events alone do not establish what changed between tests. AppLocker uses an Authenticode hash to identify the approved file.

Windows Installer, Script and Packaged App Rules

Windows Installer

Action	User	Name	Conditi
Allow	Everyone	(Default Rule) All Windows Installer files in %systemdri...	Path
Allow	BUILTIN\Ad...	(Default Rule) All Windows Installer files	Path

Windows Installer folder allowance and administrator default.

Script

Action	User	Name	Condition
Allow	Everyone	(Default Rule) All scripts located in the Windows folder	Path
Allow	BUILTIN\Ad...	(Default Rule) All scripts	Path

Windows-folder scripts and administrator default.

Packaged App

Action	User	Name
Allow	Everyone	(Default Rule) All signed packaged apps

The captured default allows all signed packaged apps.

Collection scope

Each collection has its own rules. Executable rules alone do not define script, installer or packaged-app permissions.

Lab evidence

The signed packaged-app default is a broad allowance. This capture does not demonstrate a restricted app-by-app policy.

AppLocker Limitations and Bypass Surface

Production context – not implemented or tested in this lab

- **DLL side-loading:** the DLL collection is disabled by default. Without it, executable allow rules do not check the DLLs an approved application loads.
- **Trusted binaries (LOLBins):** allowed signed system tools can provide proxy execution paths. A trusted signature alone does not make every use safe.
- **Writable paths:** users who can write to an allowed folder may place unapproved software there. Directory permissions are part of the trust decision.
- **Coverage gaps:** some interpreted content and host processes fall outside direct AppLocker checks. Complement application rules with endpoint monitoring and least privilege.

Microsoft Learn: Working with AppLocker rules; Security considerations for AppLocker.

Troubleshooting and Lessons Learned

1. System stability

Removing essential Windows allowances blocked core processes and broke the shell. Safe Mode recovery was required.

2. Rule precision

Path, Publisher and Hash conditions solve different application-control problems and have different maintenance trade-offs.

3. Validation

Execution tests and AppLocker event logs were used to confirm that policy changes were actually enforced.

Practical takeaway

Introduce allow-listing changes incrementally, test them with non-administrative users, preserve a recovery path, and verify the result in AppLocker logs before broader enforcement.

Production Security Considerations

Lab approach

- Windows Server 2022 test system
- Local Group Policy configuration
- TestUser for enforcement validation
- Default-deny behaviour
- Publisher, Path and File Hash conditions
- Manual functional and event-log verification

Production improvements to consider

- Stage new policies in Audit only mode first
- Deploy centrally through domain Group Policy or MDM
- Scope policies by OU and security group
- Monitor AppLocker events centrally
- Prefer maintainable signed-software rules where practical
- Review writable paths, exceptions and trusted system binaries

Important distinction

These are production recommendations identified after the exercise. Central domain deployment, audit-mode staging and the stronger controls discussed here are not claimed as features implemented in the original lab.

Project Summary

Implemented

- AppLocker enforcement through local Group Policy
- Default Windows and administrator rules
- User-specific path-based Deny rule
- Restrictive File Hash allow-listing
- Publisher, Path and Hash multi-tier policy
- Installer, Script and Packaged App collections

Validated

- Blocked execution for TestUser
- Expected default-deny behaviour
- AppLocker Event Viewer evidence
- Recovery from an over-restrictive policy

Result

The lab demonstrated practical Windows application control with several AppLocker rule types and a progressively more restrictive policy model.

The strongest lesson was not simply how to create a rule, but how to balance application trust, user scope, system stability, validation, maintenance and recovery.